WELCOME TO MY CLASS!!!

- Today's agenda
  - Introducing the instructor
  - Course overview and brief idea about syllabus
  - C review (as much as we can)
    - Pointer
    - Array, 2d array
    - String
    - Structure
    - File IO

C programming review

Dr. Tanvir Ahmed

Starting with Fun!



Agenda

- In this lecture, we will review some advance topics in C programming language
- Generally, it is assumed that you already have some idea about these topics and a review will help you to recall them.
- These topics are very important to understand the subsequent advance topics and data structures that we will learn in this course.
- If you think that you are not familiar with them, you should spend good amount of time to review them by this week.
- I have posted some recording on structure topics in webcourses. Consider watching them to have a very good overview of structure and structure with pointers
- Also, I have posted some notes on strings, arrays, file i/o, etc.

Agenda

- Data and Memory
- Pointer review
  - Declaring, dereferencing, pointer of pointer
  - Function call by value and reference
- Arrays review
  - Array concepts
  - Array and pointers
  - Passing array to a function
  - 2D array
- String review
- Structure Review
- File I/O review

Data and Memory

Data and Memory

- **Some very basic definitions:**
- **bit - a single zero or one. The word comes from the contraction of "binary digit.**
- **byte - eight consecutive bits. Can store a char.**
- **word - the size of this depends on the computer, but they are almost always either 2 or 4 bytes. Usually stores an int.**

- **You can imagine memory as a long array of numbered cells.**
- **For example, if we had 4 megabytes, then we would have memory locations (each storing a byte) numbered from 0 to $4x2^{20}-1$.**

Data and Memory and size of a variable

- **If we execute the line**: char **ch = 'A'**
- **and the variable ch happened to be stored in memory location 1000, our picture would look like this:**
-

…….								
1002								
1001								
1000	**Binary of ascii of 'A'**							
……								

- **An integer would be stored over 4 consecutive bytes, and the address of the variable storing the integer would be the first memory address the variable was stored in.**
- **How to get a size of a variable or type?**
- **In certain situations, it's useful to know the size of a particular variable or type/struct.**
- **The following expression returns the desired information:**
- **sizeof(int)** **or sizeof (x) //if x is a variable**

Pointer Review

(We will briefly go through pointer review. If you need more explanation, please review the uploaded slide on pointers)

Pointers

- A pointer can store the memory address of a variable of a specific data type.
- Why specific data type? Because different type stored differently and takes different size in memory.
- Example of declaring a pointer:

```
int *p; //You have to put * before the variable name. Like any uninitialized variable,
here p is pointing to a garbage address as it is not initialized yet.
```

- Another example bellow:

```
int *pointer1 = &x; // Example of both declaration and initialization in the same line
```



- **The above statement also can be written as two lines**
- `int *pointer1;`
- `pointer1 = &x;`

Basic Pointer Operators

- **We can declare a pointer as follows:**
- **int *p;**

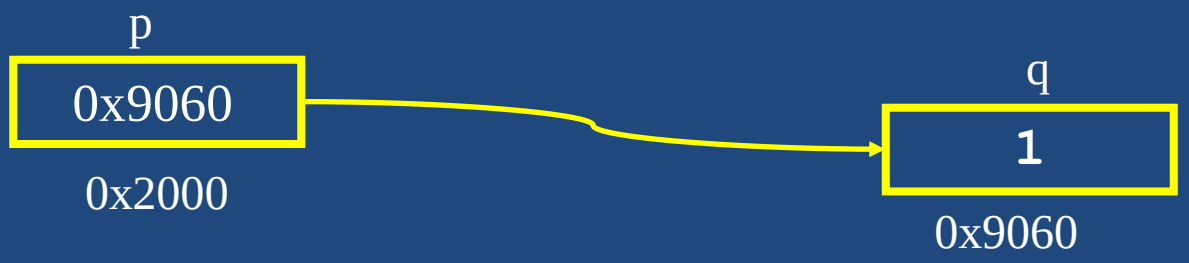
- **There are two key pointer operators:**

- ***p: Dereferences a pointer, returns the value stored at the memory location p is pointing to. // note: this *p is not in the pointer declaration line shown above;**
- **&x: Returns the address of where the variable x is stored.**

- ***According to the example in the last slide, if we write `printf("%d", *p)` it will print x (not the address of x).***

De-referencing Example

Consider this few lines of codes bellow, It will print 1:




```
int *p , q;
p=&q;
printf("%d", *p); //this will print the value of q
```

- **Why it is printing the value of q?**
  - **Because, we are de-referencing p by writing *p**
  - **The *p in the printf means, the variable it is pointing to. As p is pointing to q, *p means actually q.**
  - **Now you know the differences between *p in the first line and *p in the third line**

Summary of using * and &

- To declare a pointer add an * in front of its name.
- To obtain the address of a variable use **&** in front of its name.
- To obtain the value of a variable pointed by a pointer, use * in front of a pointer's name.

Pointer example

```
int main()
{
    int x, y;
    int *p1, *p2;
    x = -42;
    y=163;
    p1 = &x;
    p2 = &y;
    printf("p1 = %p *p1 = %d \n", p1, *p1);
    printf("p2 = %p *p2 = %d \n", p2, *p2);
    printf("---------\n");

    *p1 = 17; // dereferencing. It means put 17 to the address
where p1 is pointing to. Kind of x = 17
    printf("p1 = %p *p1 = %d \n", p1, *p1);
    printf("---------\n");

    p1 = p2; //p1 is not pointing to x anymore. Now pointing to y
    printf("p1 = %p *p1 = %d \n", p1, *p1);
    printf("---------\n");

    p2 = &x;
    printf("---------\n");
    printf("p2 = %p *p2 = %d \n", p2, *p2);

    *p1 = *p2;
    printf("p1 = %p *p1 = %d \n", p1, *p1);
    printf("---------");
    return 0;
}
```

```
output:
p1 = 0x7ffce97158f8 *p1 = -42
p2 = 0x7ffce97158fc *p2 = 163
---------
p1 = 0x7ffce97158f8 *p1 = 17
---------
p1 = 0x7ffce97158fc *p1 = 163
---------
---------
p2 = 0x7ffce97158f8 *p2 = 17
p1 = 0x7ffce97158fc *p1 = 17
---------
```

Testing pointer is pointing something meaningful (Dealing with segmentation fault)

- If a pointer is not initialized to any address, dereferencing that pointer might result in garbage value, or can create **segmentation fault/run-time error** as your code is trying to access a memory which is not allocated to your program. In that case we can use the following technique to prevent such potential error.
- We can assign NULL to a pointer.
- `int *aPtr = NULL;`
- **It's important not to dereference a null pointer. Doing so will lead to a run-time error.**
- However, before dereferencing we can check it is NULL or not:
  - `if (aPtr)  //it checks if aPtr is not NULL.`
    `{ write statements that can use aPtr as it is safe to do so}`
- The above line checks if aPtr is NULL or not.
  - You could also write: `if (aPtr!= NULL) //this is same as if (aPtr)`
- Note that it is programmer's responsibility to assing NULL values when necessary

Why to use pointer

- **C programming language WANTED the programmer to have the ability to directly manipulate memory.**

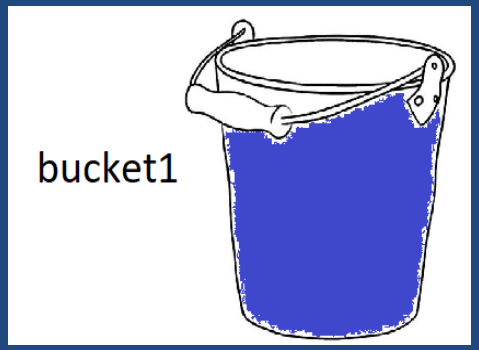
- **1) Allow you to refer to a large data structure in a compact way.**
- **2) Allow you to share data efficiently among functions.**
- **3) Allow you to dynamically reserve memory.**
- **4) Allow you to record relationships between data.**

Pass by value

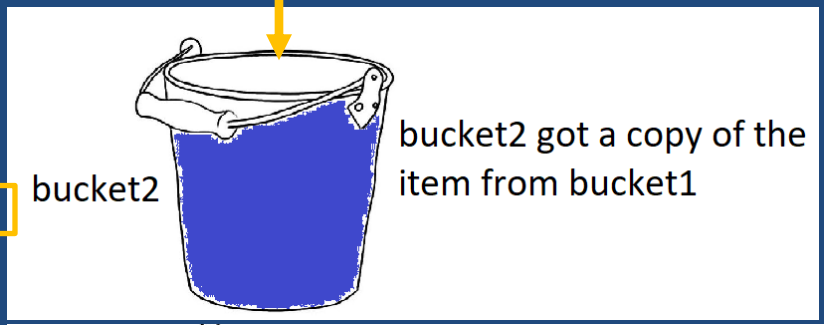
```
#include <stdio.h>
#define blue 1
#define red 2
void callbyvalue(int bucket2);

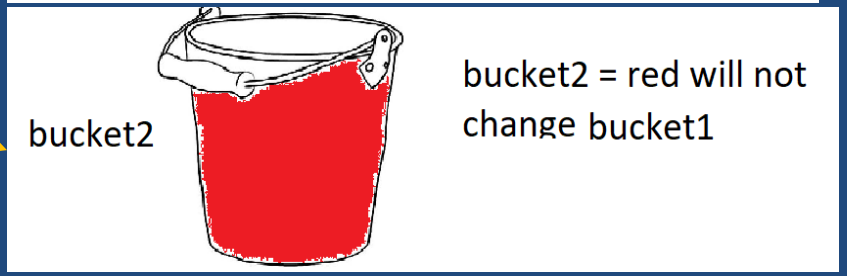
int main()
{
    int bucket1 = blue;
    callbyvalue(bucket1);
    printf("%d", bucket1);
    return 0;
}

void callbyvalue(int bucket2)
{
    bucket2 = red;
}
```



callbyvalue(bucket1);//sending a copy of items in bucket1



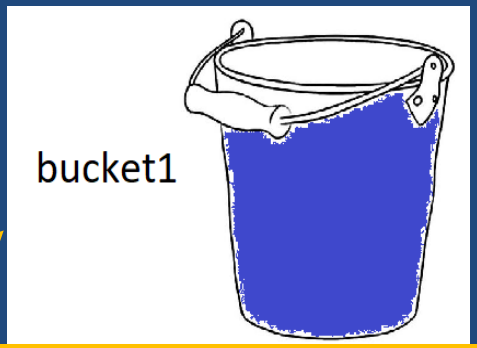


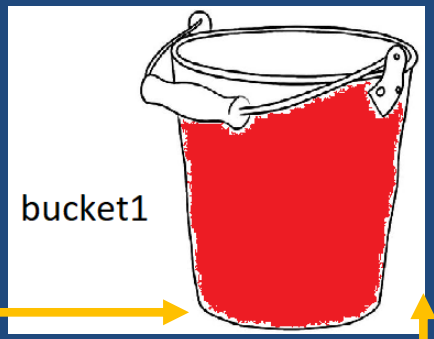
Pass by reference

```
#include <stdio.h>
#define blue 1
#define red 2
void callbyreference(int *bucket2);

int main()
{
    int bucket1 = blue;
    callbyreference(&bucket1);
    printf("%d", bucket1);
    return 0;
}

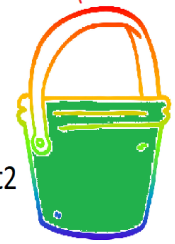
void callbyreferenceint *bucket2)
{
   *bucket2 = red;
}
```





callbyreference(&bucket1); sending locatin of bucket1

callbyreference(int *bucket2) //receiving the location of bucket1 into bucket2



bucket2 is a pointer, storing the location of bucket1



*bucket2 = red will change the content of bucket1 (we have to use * to refer back to bucket1

Pointers and Function Arguments Another example

- **Call by value**
- **Call by reference**



Solution 1

```
#include <stdio.h>

void fakeSwap(int a, int b)
{
    int tmp;

    tmp = a;
    a = b;
    b = tmp;
}

int main()
{
    int x = 1, y = 2;

    fakeSwap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Solution 1

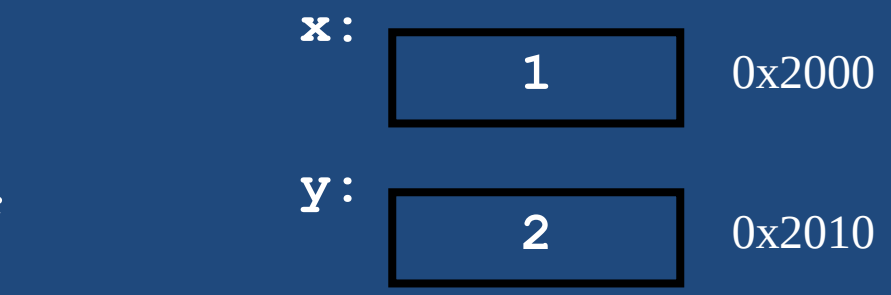
```
#include <stdio.h>

void fakeSwap(int a, int b)
{
    int tmp;

    tmp = a;
    a = b;
    b = tmp;
}

int main()
{
    int x = 1, y = 2;

    fakeSwap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```



```
#include <stdio.h>

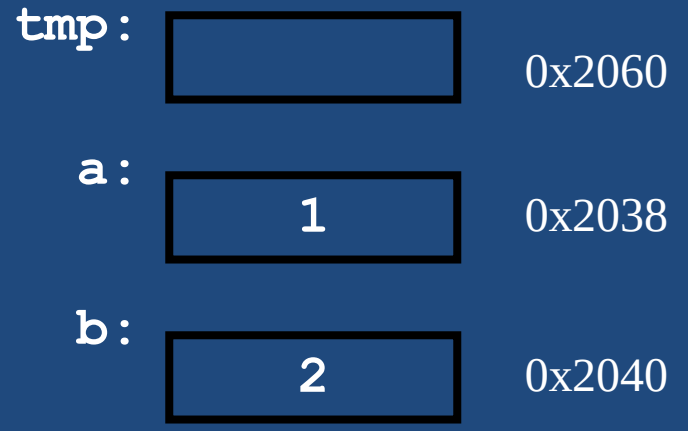
void fakeSwap(int a, int b)
{
    int tmp;

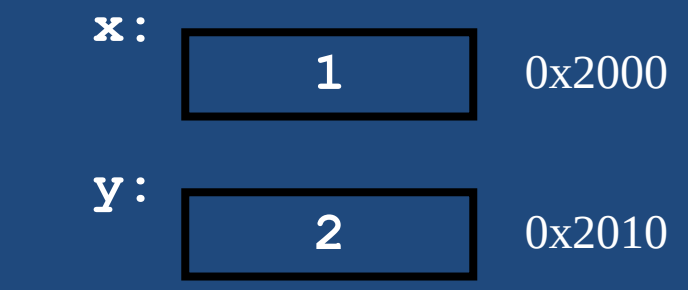
    tmp = a;
    a = b;
    b = tmp;
}

int main()
{
    int x = 1, y = 2;

    fakeSwap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Solution 1





```
#include <stdio.h>

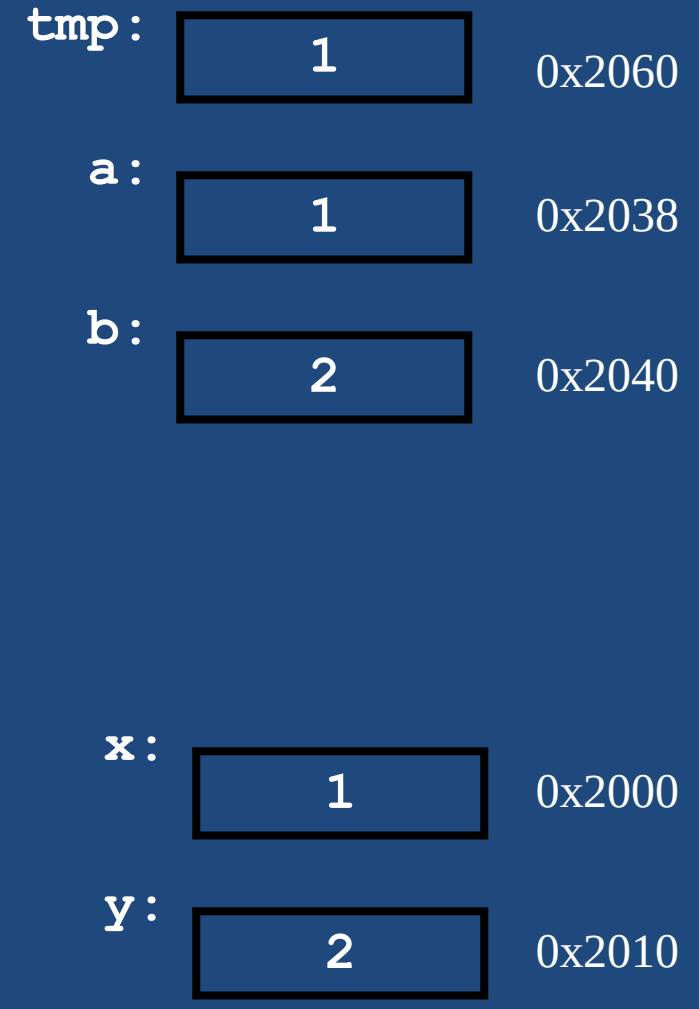
void fakeSwap(int a, int b)
{
    int tmp;

    tmp = a;
    a = b;
    b = tmp;
}

int main()
{
    int x = 1, y = 2;

    fakeSwap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Solution 1



```
#include <stdio.h>

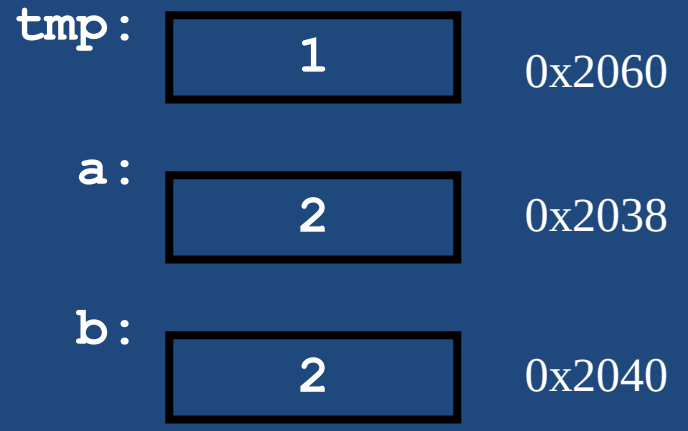
void fakeSwap(int a, int b)
{
    int tmp;

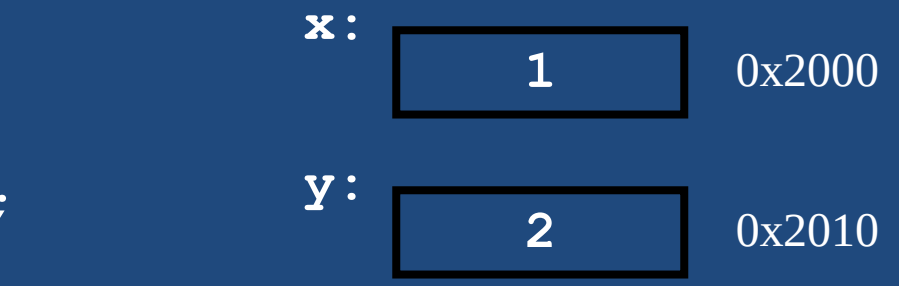
    tmp = a;
    a = b;
    b = tmp;
}

int main()
{
    int x = 1, y = 2;

    fakeSwap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Solution 1





Solution 1

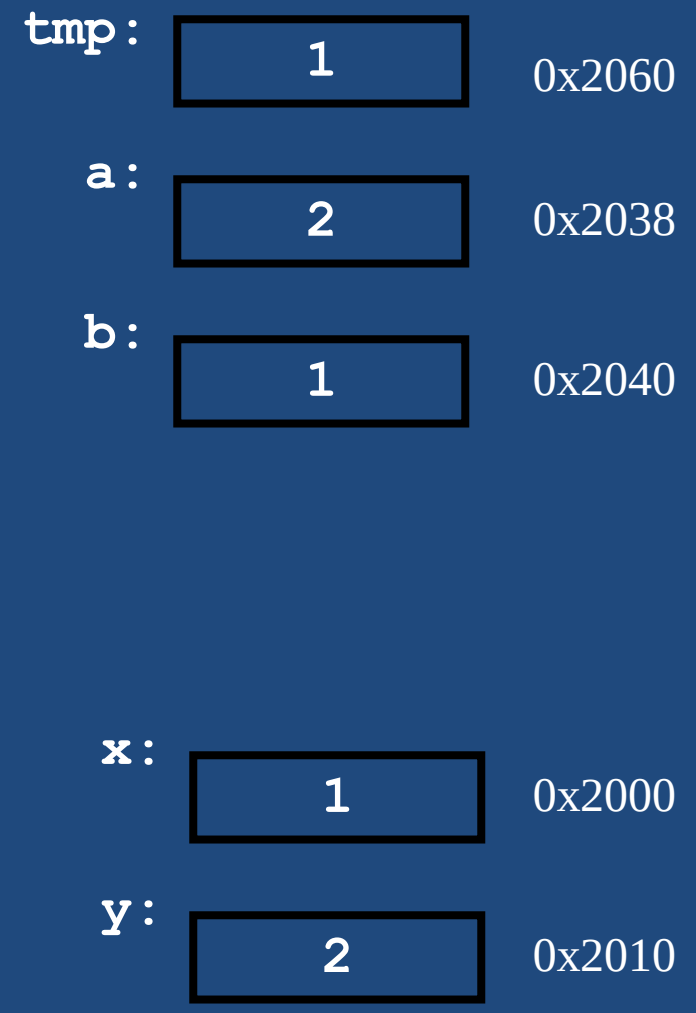
```
#include <stdio.h>

void fakeSwap(int a, int b)
{
    int tmp;

    tmp = a;
    a = b;
    b = tmp;
}

int main()
{
    int x = 1, y = 2;

    fakeSwap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```



Solution 1

```
#include <stdio.h>

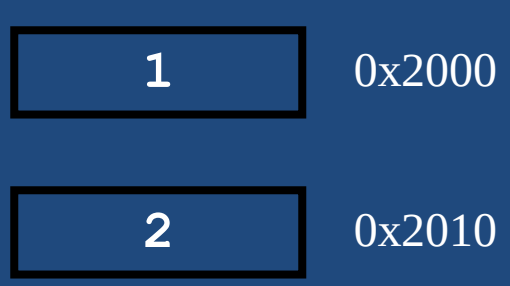
void fakeSwap(int a, int b)
{
    int tmp;

    tmp = a;
    a = b;
    b = tmp;
}

int main()
{
    int x = 1, y = 2;

    fakeSwap(x, y);
    printf("%d %d\n", x, y);
    return 0;
}
```





Solution 2

```
#include <stdio.h>

void trueSwap(int* a, int* b)
{
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main()
{
    int x = 1, y = 2;

    trueSwap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Solution 2

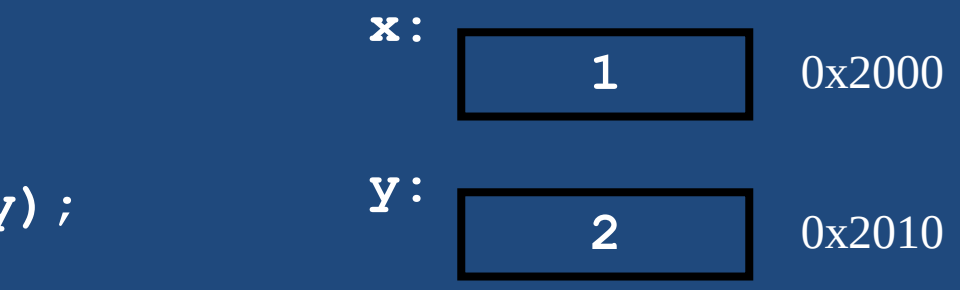
```
#include <stdio.h>

void trueSwap(int* a, int* b)
{
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main()
{
    int x = 1, y = 2;

    trueSwap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```



```
#include <stdio.h>

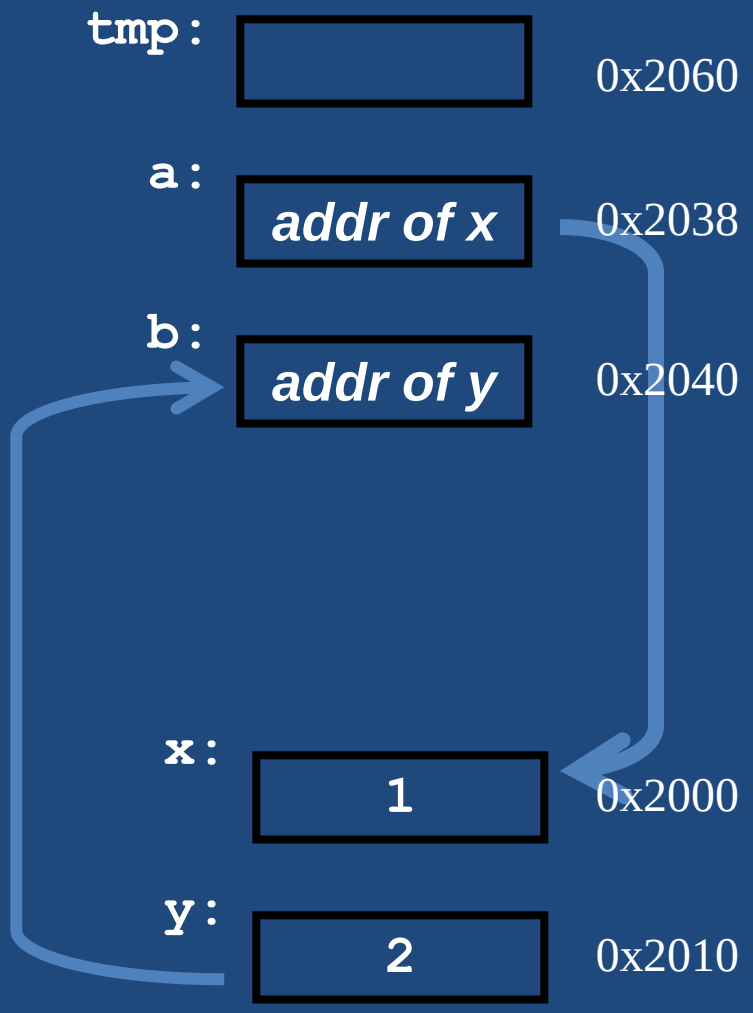
void trueSwap(int* a, int* b)
{
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main()
{
    int x = 1, y = 2;

    trueSwap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Solution 2



Solution 2

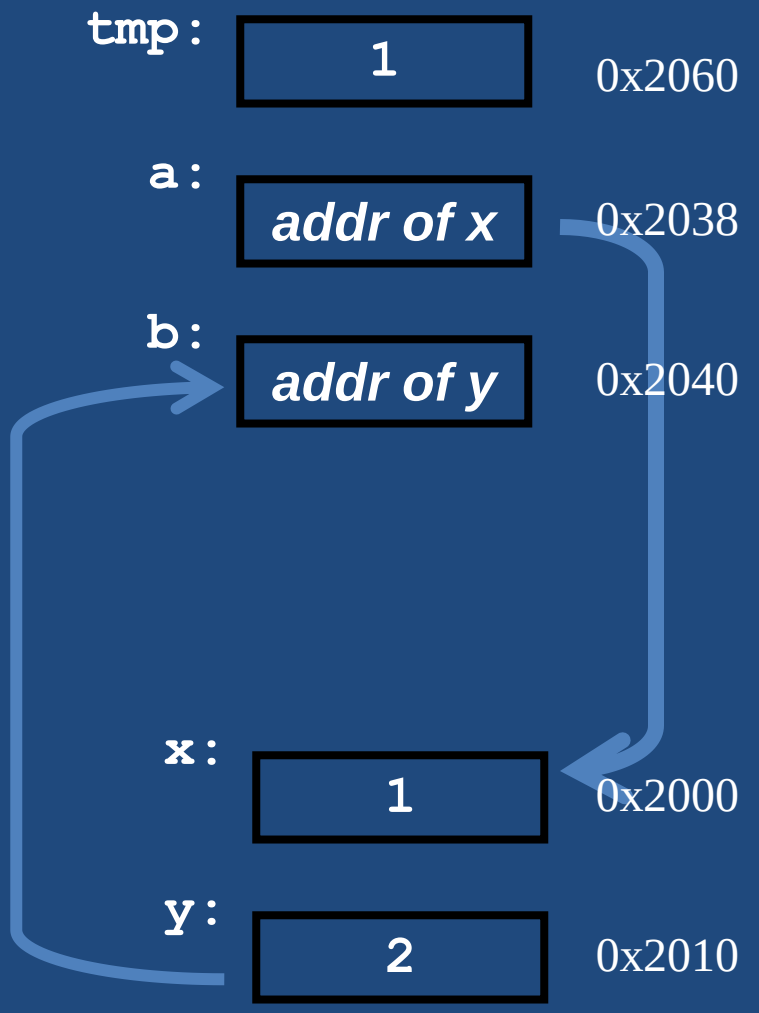
```
#include <stdio.h>

void trueSwap(int* a, int* b)
{
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main()
{
    int x = 1, y = 2;

    trueSwap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```



Solution 2

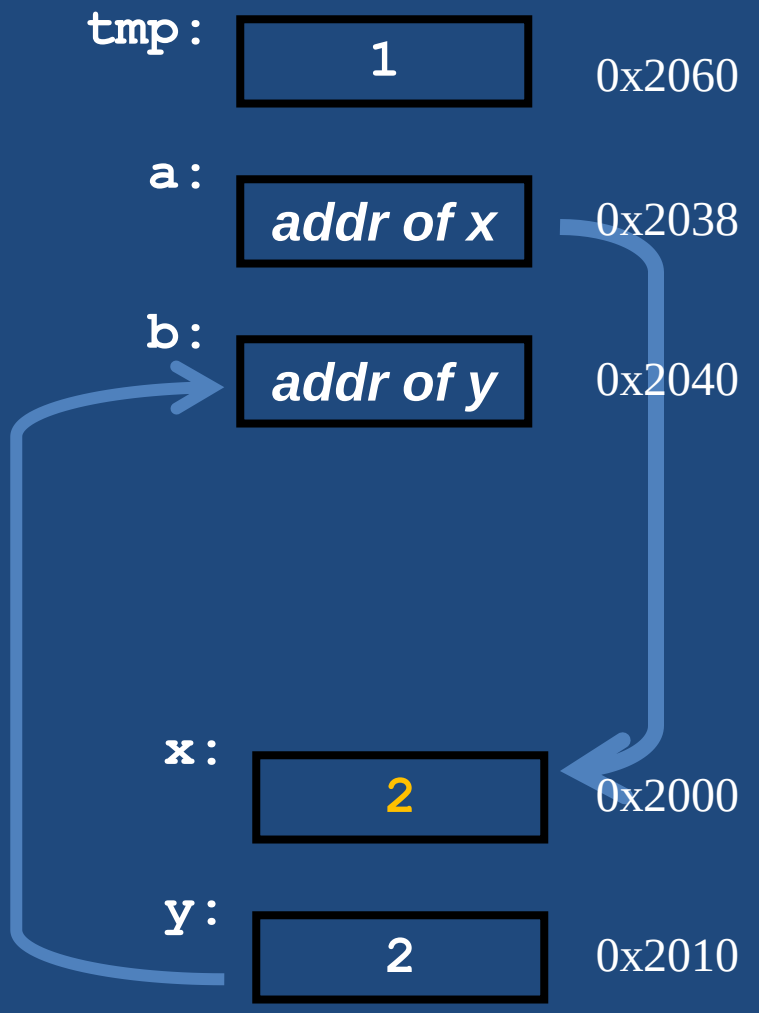
```
#include <stdio.h>

void trueSwap(int* a, int* b)
{
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main()
{
    int x = 1, y = 2;

    trueSwap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```



Solution 2

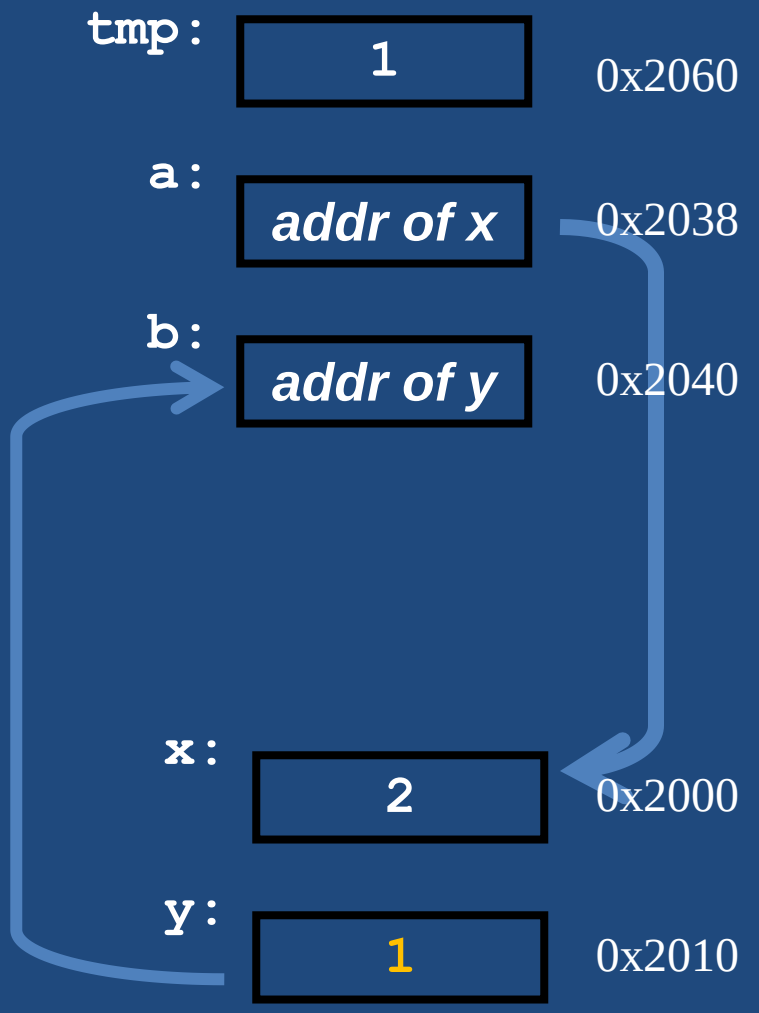
```
#include <stdio.h>

void trueSwap(int* a, int* b)
{
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main()
{
    int x = 1, y = 2;

    trueSwap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```



Solution 2

```
#include <stdio.h>

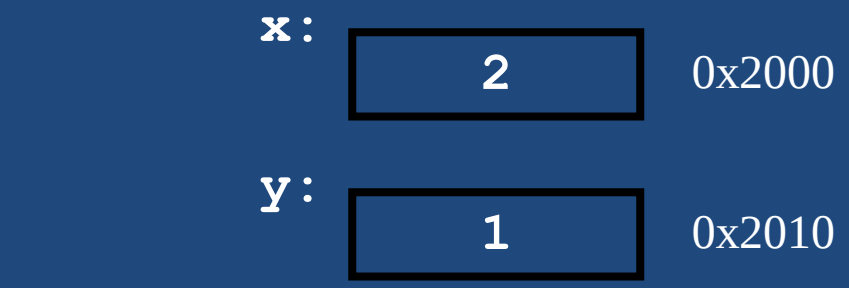
void trueSwap(int* a, int* b)
{
    int tmp;

    tmp = *a;
    *a = *b;
    *b = tmp;
}

int main()
{
    int x = 1, y = 2;

    trueSwap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```





Call by value and call by reference example conclusion

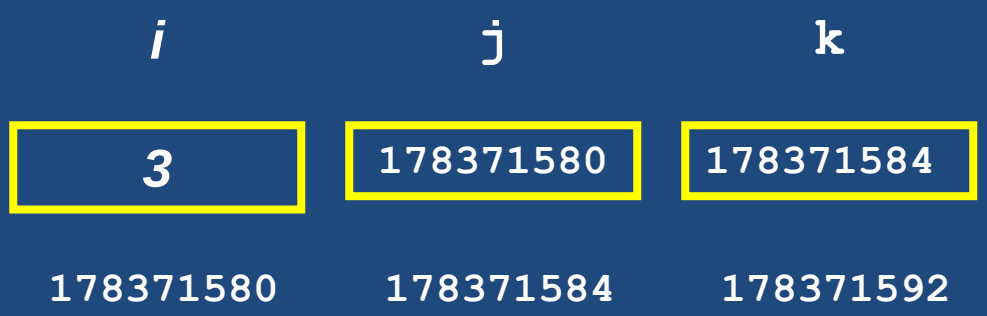
- The first solution used call by value method.
  - It just copied the values from x, y into a, b respectively
  - Change in a and b did not affect x and y
- The second solution used call by reference
  - We used pointer a and be to hold the address of x and y
  - So in the function we actually changed the value within the address of x and y.
  - So after completing the function call, the value of x and y have changed

Pointer of Pointer

- The concept of pointers can be further extended.
- Pointer, we know is a variable that contains address of another variable.
- Now this variable itself might be another pointer.
- Thus, we now have a pointer that contains another pointer's address.
- The following example should make this point clear.

```
int main( )
{
    int i = 3, *j, **k ;
    j = &i ;
    k = &j ;// K has the capability to hold address of pointer
    printf ( "\nAddress of i = %u", &i ) ; //print address of i
    printf ( "\nAddress of i = %u", j ) ; //print address of i
    printf ( "\nAddress of i = %u", *k ) ; //print address of i
    printf ( "\nAddress of j = %u", &j ) ; //print address of j
    printf ( "\nAddress of j = %u", k ) ; //print address of j
    printf ( "\nAddress of k = %u", &k ) ; //print address of k
    printf ( "\nValue of j = %u", j ) ; //print value of j, which is address of i
    printf ( "\nValue of k = %u", k ) ; //print value of k, which is address of j
    printf ( "\nValue of i = %d", i ) ; //print 3
    printf ( "\nValue of i = %d", * ( &i ) ) ; //print 3
    printf ( "\nValue of i = %d", *j ) ; //print print 3
    printf ( "\nValue of i = %d", **k ) ; //print 3. As *k = j, and *j = i, i=3
    return 0;
}
```

```
//output:
Address of i = 178371580
Address of i = 178371580
Address of i = 178371580
Address of j = 178371584
Address of j = 178371584
Address of k = 178371592
Value of j = 178371580
Value of k = 178371584
Value of i = 3
Value of i = 3
Value of i = 3
Value of i = 3
```



Arrays

Arrays

- An array is a collection of variables, with:
- 1*) The items are stored in consecutive memory locations*
- 2) *All the variables in an array are of the same type.*

- We define an array as follows: int data[10];
- Here the size of the array is 10.
- To index a particular variable/element of the array do as follows:

  data[4] = 120;
- The first index is data[0] and last index is data[9]
- The items are: data[0], data[1], data[2], …., data[9]
- They contains garbage data if not initialized.
- Another way of declaring and initializing array:

  int data[] = {87, 99, 75, 88, 93, 56, 77, 84, 89, 79};

Pointers and Arrays

- Array elements are stored in consecutive memory locations
- So, if you know the first address of the array, you can easily access the remaining array elements!
- See the following example array.
- int A[8]

Index	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]
Value	5	6	8	23	22	9	90	4
address	1000	1004	1008	1012	1016	1020	1024	1028

Look at the address!
Increasing by 4 as each
int takes 4 bytes

Pointers and Arrays

```
int array[size];
int* pPtr = array + i; //let's say i is an index of the array
int* qPtr = array + j; //let's say j is an index of the array
```

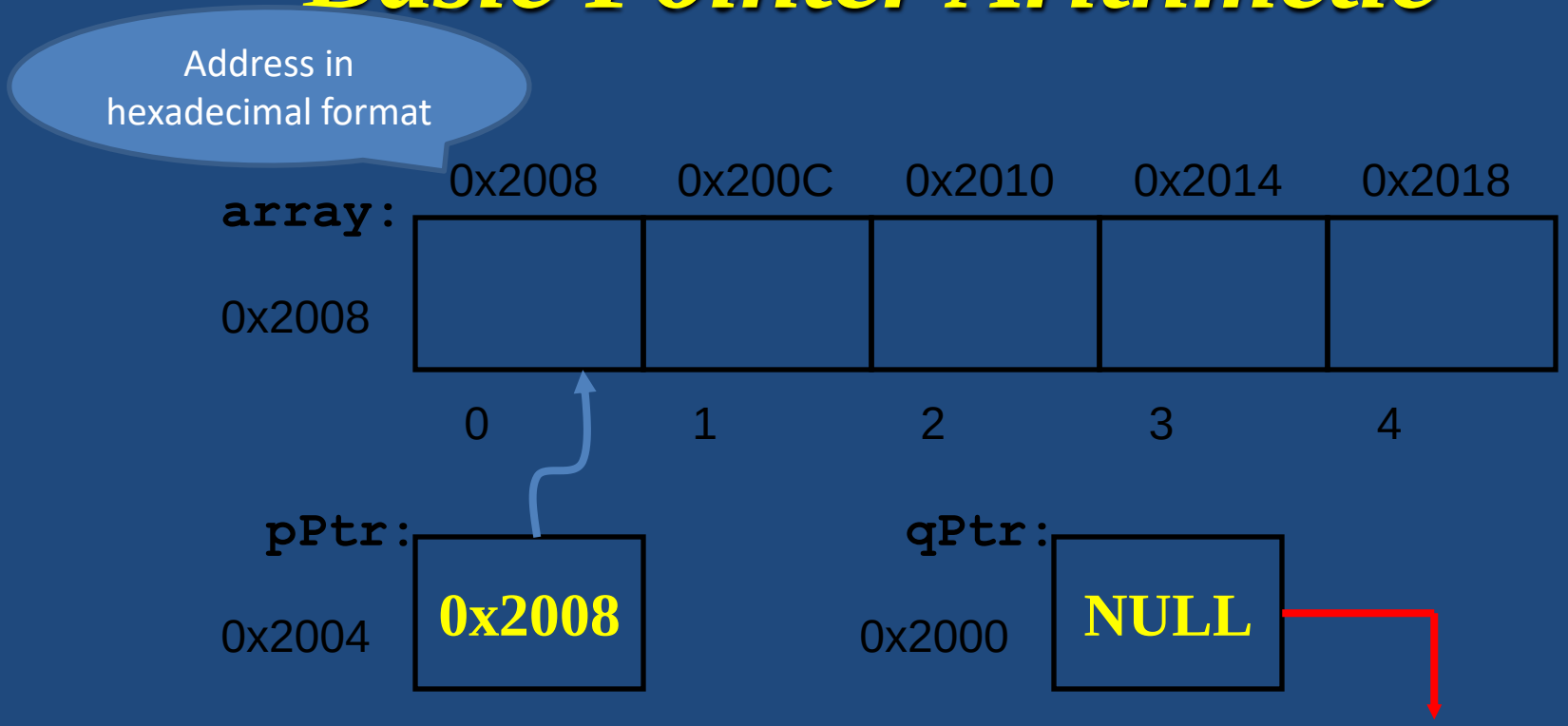
- **The name `array` is equivalent to `&array[0]`**
- It means, name of an array is a pointer
- **`pPtr++`** increments **`pPtr`** to point to the next element of `array`.
- `pPtr += n` increments `pPtr` to point to n elements beyond where it currently points.
- `pPtr-qPtr` equals `i-j`.
- Thus, we can see that a pointer arithmetic depends on the type.
- The increment or decrement will jump to **4 bytes** if the pointer is integer pointer.

Pointers and Arrays

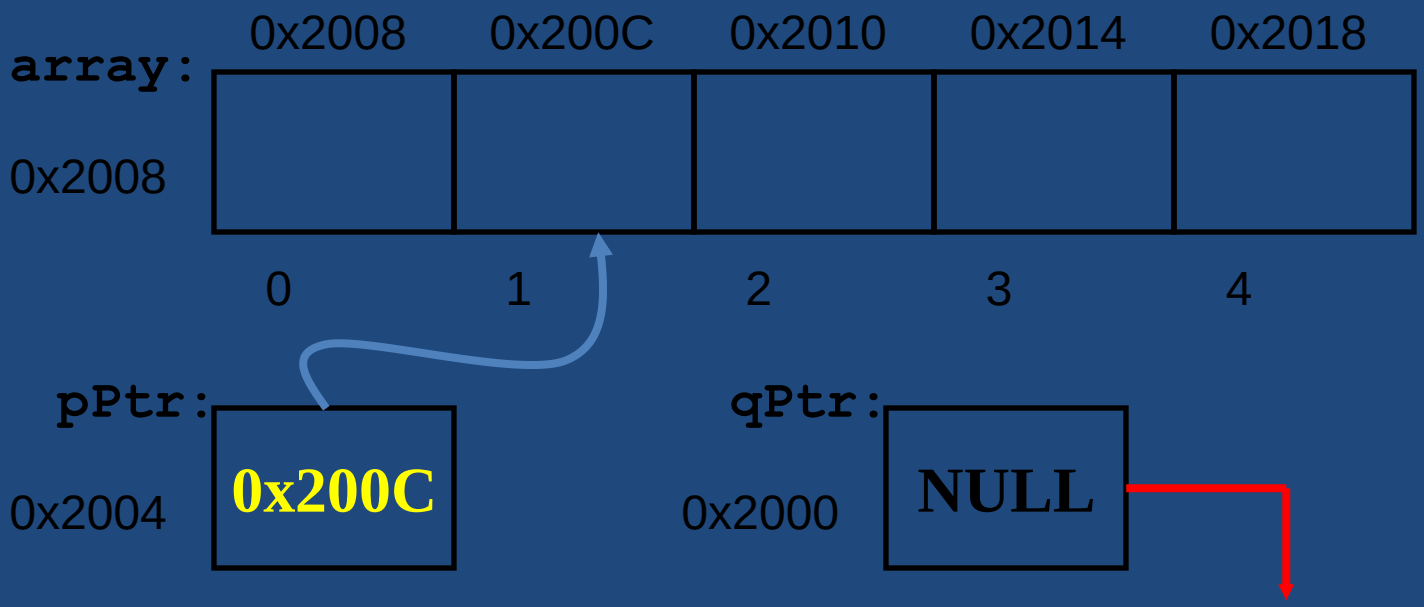
Type `array[`*size*`];`

- `array[0]` **is equivalent to** `*array`
- `array[n]` **is equivalent to** `*(array + n)`
- How?
  - **We know array name is equivalent to the address of first element**
  - **So, `array` is same as `&array[0]`**
    - **So, `*array` is dereferencing and getting `array[0]`**
  - **`(array+n)` is the address of nth element,**
    - **So, `*(array+n)` is equivalent to `array[n]`**

Basic Pointer Arithmetic



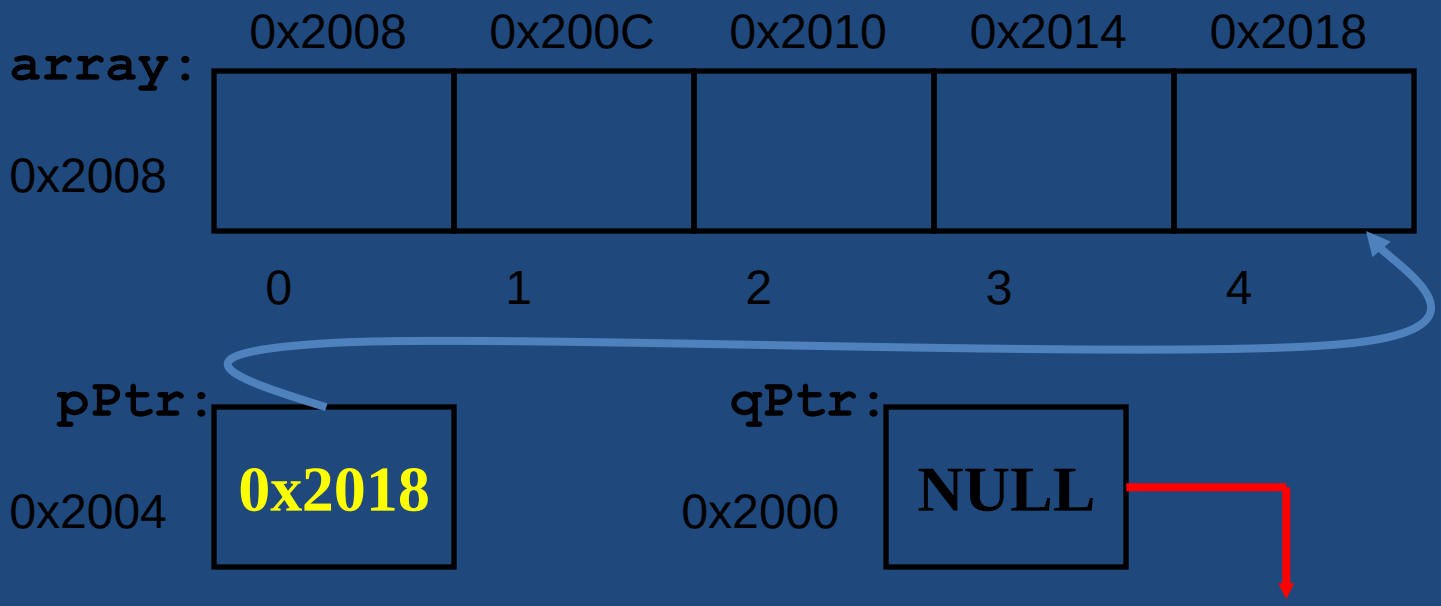

```
float  array[5];
float* pPtr = array;
float* qPtr = NULL;
```




```
float  array[5];
float* pPtr = array;
float* qPtr = NULL;
```

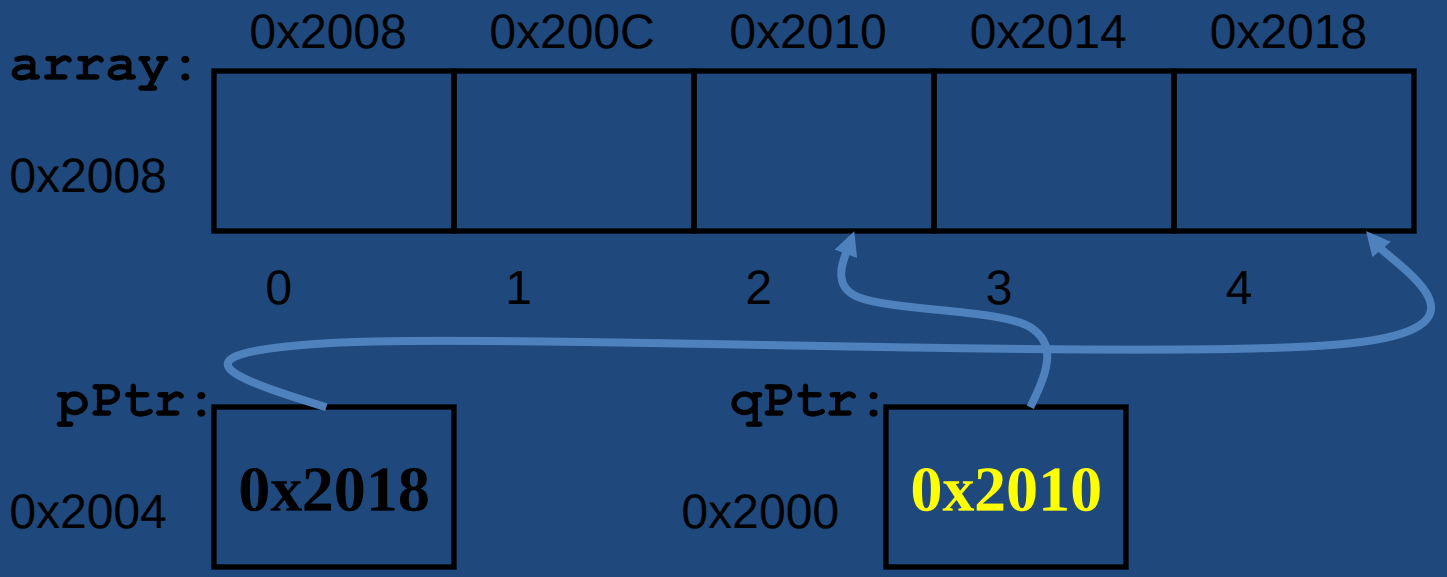
```
pPtr++; /* pPtr now holds the address: &array[1] */
```

Note: pPtr is not becoming 2008 to 2009. It is jumping to the next int as it is an int pointer. It is increased by 4 as an int takes 4 bytes



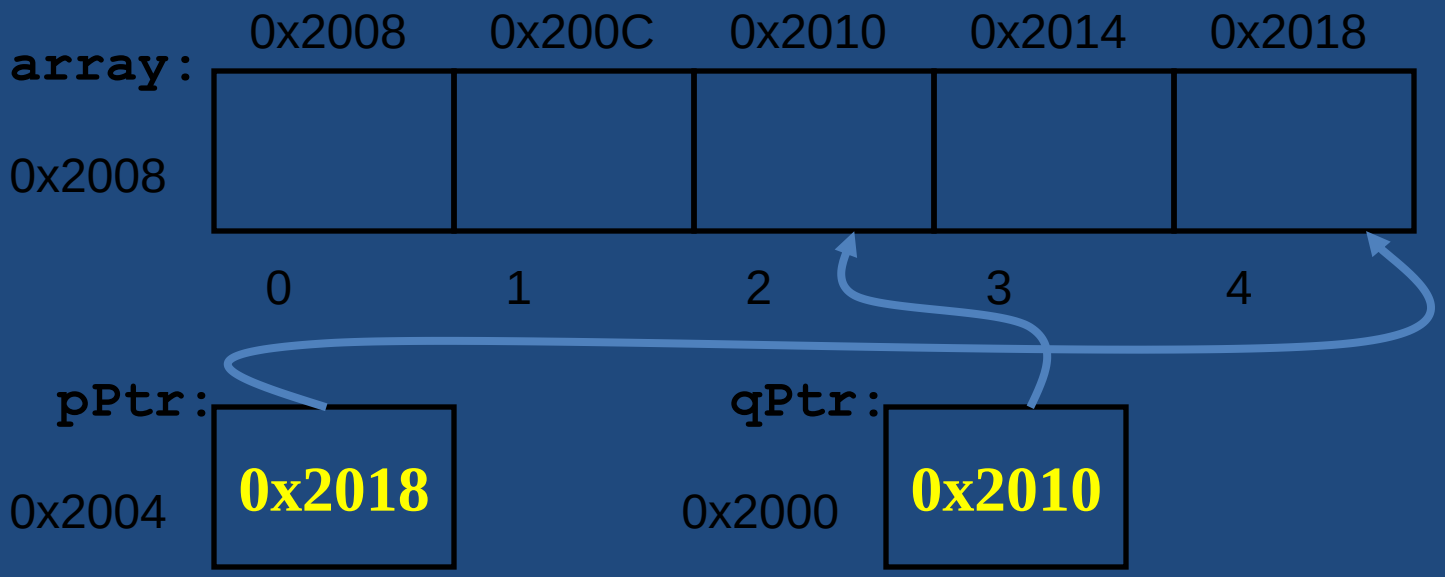

```
float  array[5];
float* pPtr = array;
float* qPtr = NULL;


pPtr++; /* pPtr = &array[1] */
pPtr += 3; /* pPtr now holds the address: &array[4] */
```




```
float  array[5];
float* pPtr = array;
float* qPtr = NULL;


pPtr++; /* pPtr = &array[1] */
pPtr += 3; /* pPtr = &array[4] */
qPtr = array + 2; /*qPtr now holds the address &array[2]*/
```




```
float  array[5];

float* pPtr = array;

float* qPtr = NULL;


pPtr++; /* pPtr = &array[1] */

pPtr += 3; /* pPtr = &array[4] */

qPtr = array + 2; /* qPtr = &array[2] */
printf("%d\n", pPtr-qPtr); //the answer is 2. There are
                           two integers between them.
```

Example: traversing through array using pointer

```
int main()
{
    int data[] = {87, 99, 75, 88, 93};
    int *p;
    int i;
    for(i = 0; i<5; i++)
        printf("address: %p, value: %d, test: %d\n", data+i,
                                                *(data+i), *data+i);


    printf("--------using p------------\n");
    p = data; // p is now &data[0]
    for(i = 0; i<5; i++)
    {
        printf("address: %p, value: %d \n", p, *p);
        p++;
    }

    return 0;
}
```

```
output:
address: 0x7ffd8ec91e10, value: 87, test: 87
address: 0x7ffd8ec91e14, value: 99, test: 88
address: 0x7ffd8ec91e18, value: 75, test: 89
address: 0x7ffd8ec91e1c, value: 88, test: 90
address: 0x7ffd8ec91e20, value: 93, test: 91
--------using p------------
address: 0x7ffd8ec91e10, value: 87
address: 0x7ffd8ec91e14, value: 99
address: 0x7ffd8ec91e18, value: 75
address: 0x7ffd8ec91e1c, value: 88
address: 0x7ffd8ec91e20, value: 93
```

It is just adding i to data[0].
**data is dereferencing data (which is &data[0])*

The values of the array also can be accessed using:
data[i]
***(i+data)**
i[data]

What is the difference between pointer and array?

- **While double values[10]; allocates 10 locations to store doubles, double *p; does not.**
- **Instead double *p; only allocates the memory to store a single location in memory.**
- **However, we CAN use a pointer to dynamically allocate memory, that we will learn later in the semester (if we get time).**

Passing Entire Array to Function

```
main( )
{
    int num[ ] = { 24, 34, 12, 44, 56, 17 } ;
    int i;
    printf("Original Array: ");
    for(i=0; i<6; i++)
       printf("%d  ", num[i]);

     incBy2( num, 6 ) ;

    printf("\nArray after calling incBy2: ");
    for(i=0; i<6; i++)
         printf("%d  ", num[i]);
}
void incBy2(int *A, int n)//also can be written as: incBy2(int A[], int n) or incBy2(int A[6], int n)
{
     int i ;
     for ( i = 0 ; i < n ; i++ )
     {
         *A = *A + 2; //in crease an a particular array element by 2
         A++; //A is jumping to the next array element's address
     }
}
```

What are we passing?
Reference or value?

!!!also can be written as: incBy2(int A[], int n) or incBy2(int A[6], int n)

```
Output:
Original Array: 24  34  12  44  56  17
Array after calling incBy2: 26  36  14  46  58  19
```

Another version-Passing Entire Array to Function (Generally we use this approach

```
main( )
{
    int num[ ] = { 24, 34, 12, 44, 56, 17 } ;
    int i;
    printf("Original Array: ");
    for(i=0; i<6; i++)
         printf("%d  ", num[i]);

     incBy2( num, 6 ) ;

    printf("\nArray after calling incBy2: ");
    for(i=0; i<6; i++)
         printf("%d  ", num[i]);
}
void incBy2(int *A, int n)//also can be written as: incBy2(int A[], int n) or incBy2(int A[6], int n)
{
     int i ;
     for ( i = 0 ; i < n ; i++ )
     {
       //even A is a pointer, you can write it like an array as you have passed an array here
         A[i] = A[i] + 2;

     }
}
```

!!!even A is a pointer, you can use array syntax with it as you have passed an array here.
We generally use this syntax as it is easy to follow like an array

```
Output:
Original Array: 24  34  12  44  56  17
Array after calling incBy2: 26  36  14  46  58  19
```

Multidimensional Arrays

- It is also possible to have array with two or more dimensions
- 2 dimensional array also called a matrix and easy to visualize in tabular form.
- An example declaration:
- int grid[4][2];
- This array as 4 rows and 2 columns. Can be seen as:

	Colum 0	Column 1
Row 0	grid[0][0]	grid[0][1]
Row 1	grid[1][0]	grid[1][1]
Row 2	grid[2][0]	grid[2][1]
Row 3	grid[3][0]	grid[3][1]

Multidimensional Arrays

- The declaration int grid[4][2]; can also be interpreted as:
- This is an array of 4 elements
  - Each element is an array of 2 elements
- So, it is kind of **4 one-dimensional** arrays.
- Example: grid[0] is an array of 2 elements
  - grid[0][0], and grid[0][1]
- The elements are laid out sequentially in memory, just like a one-dimensional array

Multidimensional Arrays

Index	grid[0][0]	grid[0][1]	grid[1][0]	grid[1][1]	grid[2][0]	grid[2][1]	grid[3][0]	grid[3][1]
Value	5	6	8	23	22	9	90	4
address	1000	1004	1008	1012	1016	1020	1024	1028

- The above table shows the memory map for the 2D array we have declared. The address and data are assumed. We consider int takes 4 bytes.
- See, grid[1][0] starts after completing full grid[0] array.

2D Array is an array of arrays

```
main( )
{
    int grid[4][2] = {
                        { 5, 6 },
                        { 8, 23 },
                        { 22, 9 },
                        { 90, 4 }
                    } ;
    int i ;
    for ( i = 0 ; i <= 3 ; i++ )
    printf ( "\nAddress of %d th 1-D array = %u", i, grid[i] ) ;
}
```

```
Output:
Address of 0 th 1-D array = 2961709872
Address of 1 th 1-D array = 2961709880
Address of 2 th 1-D array = 2961709888
Address of 3 th 1-D array = 2961709896
```

Accessing 2D array using pointer

- The following are equivalent:
- `s[2][1] //second row, first col`
- `*(s[2]+1)`
- `*(*(s+2)+1) //here 2 indicates the array number`

Accessing 2D array using pointer

```
main( )
{
    int s[4][2] = {
                    { 5, 6 },
                    { 8, 23 },
                    { 22, 9 },
                    { 90, 4 }
                } ;
    int i, j ;
    for ( i = 0 ; i <= 3 ; i++ )
    {
        printf ( "\n" ) ;
        for ( j = 0 ; j <= 1 ; j++ )
            printf ( "%d ", *( *( s + i ) + j ) ) ;
    }
}
```

```
Output:
5 6
8 23
22 9
90 4
```

2D array as parameter

- Only first subscript may be left unspecified
- `void f1(int grid[][10])  // valid`
- `void f3(int grid[][]); //invalid`
- **Generally, array sizes are also passed to functions while dealing with 1D or multi dimensional array**
- void print(int a[ ][4], int row, int col)
- **The sizes allow to iterate properly and access the elements within the ranges.**

Accessing 2D array using pointer

```
main( )
{
    int a[3][4] = {
    1, 2, 3, 4,
    5, 6, 7, 8,
    9, 0, 1, 6
    } ;

    print ( a, 3, 4 ) ;
}

display ( int *q, int row, int col )
{
    int i, j ;
    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf("%d ",*(q+i*col+j));
        printf ( "\n" ) ;
    }
    printf ("\n" ) ;
}
```

```
print ( int q[ ][4], int row, int col )
{
    int i, j ;
    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf ( "%d ", q[i][j] ) ;
        printf ( "\n" ) ;
    }
    printf ( "\n" ) ;
}
```

Our favorite one as we are very familiar with this syntax. So, in general use this one

Output of the code in the last slide

```
Output:
1 2 3 4
5 6 7 8
9 0 1 6

1 2 3 4
5 6 7 8
9 0 1 6
```

Accessing 2D array using pointer

```
main( )
{
    int a[3][4] = {
    1, 2, 3, 4,
    5, 6, 7, 8,
    9, 0, 1, 6
    } ;

    display ( a, 3, 4 ) ;
    print ( a, 3, 4 ) ;
}

display ( int *q, int row, int col )
{
    int i, j ;
    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf("%d ",*(q+i*col+j));
        printf ( "\n" ) ;
    }
    printf ("\n" ) ;
}
```

Let's see how this display function works

- *q is receiving the address of a[0][0]. It has no idea about 2D array.

- We will use this loop to go to different rows and columns.
- **i** will track row number and j will track column number

- We will use this formula to access a particular item in our 2D array:
- q+i*col+j //here col is the total number of columns in the array

- But how?:
- See, q is always pointing to the first address of the array. It is not moving.
- Adding i*col will help to jump i*col number of items. (kind of ith row)
- Then, adding j will help you to navigate different items in that particular row

Accessing 2D array using pointer

```
main( )
{
    int a[3][4] = {
    1, 2, 3, 4,
    5, 6, 7, 8,
    9, 0, 1, 6
    } ;

    display ( a, 3, 4 ) ;
    print ( a, 3, 4 ) ;
}

display ( int *q, int row, int col )
{
    int i, j ;
    for ( i = 0 ; i < row ; i++ )
    {
        for ( j = 0 ; j < col ; j++ )
            printf("%d ",*(q+i*col+j));
        printf ( "\n" ) ;
    }
    printf ("\n" ) ;
}
```

i*4 is helping us to jump to i_{th} row. (skip 4 numbers in each row)

Let's see how the display function works

- **When, i = 0 (dealing with row 0):**
  - j = 0:
    - *(q+ 0*4 + 0) = *(q+0) = 1
  - j=1:
    - *(q+0*4 +1) = *(q+1) = 2
  - j=2:
    - *(q+0*4 +2) = *(q+2) = 3
  - j=3:
    - *(q+0*4 +3) = *(q+1) = 4

- **When, i = 1 (dealing with row 1):**
  - j = 0:
    - *(q+ 1*4 + 0) = *(q+4) = 5
  - j=1:
    - *(q+1*4 +1) = *(q+5) = 6
  - j=2:
    - *(q+1*4 +2) = *(q+6) = 7
  - j=3:
    - *(q+1*4 +3) = *(q+7) = 8
  - Try for i = 2!

Strings

Strings review

- String is a char array
- Sequence of zero or more characters terminated by null (‘\0’) character
- Note that ‘\0’ terminates the string but it is not part of the string.
- strlen() function returns the length of a string. null is not part of the length
- there are many useful functions for strings are available at string.h

Strings review

- char s[10] = “cat”;
- Here s is an string and 10 bytes are allocated for the string or array.

0	1	2	3	4	5	6	7	8	9
c	a	t	‘\0’						

- Also can be declared using pointer:

  char *s = “cat”; //but you cannot change the content in this case. We have to use dynamic memory allocation to work on it.

- You can access each character in the string like an array.
- You can pass string to a function in the same way you pass an array.

Strings review

- You can use scanf and gets method for taking string input
- Be careful while using scanf to read string as it will skip anything after space.

```
int main()
{
    char name[20];
    printf("Enter name: ");
    scanf("%s", name);
    printf("Your name is %s.", name);
    return 0;
}
```

```
Output:
Enter name: Nusair Ahmed
Your name is . Nusair //see it failed to keep Ahmed
in the string
```

Strings review

- gets and puts methods for string.
- If you use gets, it can take string with spaces.

```
#include <stdio.h>
int main()
{
    char name[30];
    printf("Enter name: ");
    gets(name);      // read string
    printf("Name: ");
    puts(name);     // display string
    return 0;
}
```

```
Output:
Enter name: Nusair Ahmed
Name: Nusair Ahmed
```

Alternative of gets() method for string input

- gets() method is dangerous as it does not care about the size of the char array and can takes more than the size. As a result your program can crash and become unsafe.
- Alternative:

```
char name[30];
printf("Enter name: ");
scanf("%[^\n]s",name);
```

- **you can also use:** `fgets(name, sizeof(name), stdin);`
- ***In case of fgets, it also keep the '\n' in the string. The following line will remove that '\n'***
- `name[strcspn(name, "\n")] = 0;`
- **See uploaded example code: fgets_DifferentScanf.c**
- **You will also get explanation in the code**

Problems with getting string input after another input

```
int main()
{
	int id;
	char name[30];
	char ch, c;
	while(1){
		printf("\nMenu, e: enter, x: exit: ");
	    scanf("%c",&ch);
		if(ch=='x'){
			printf("\n<<<<<<<EXIT>>>>>>>\n\n\n");
			break;
		}
		if(ch=='e'){
			printf("\nEnter student id: ");
			scanf("%d",&id);
			printf("\nEnter student name: ");
			scanf("%[^\n]s",name);
			printf("Your entered %d %s", id, name);
		}
	}
	return 0;
}
```

Problems with getting string input after another input

- In the example shown at previous slide, the string input after the integer input will be skipped.
  - The reason is when you press the enter key after providing an integer input, the enter key is passed to the next string input and the console does not wait to take an string input as it already found the input.
  - To avoid this types of problem, use the following loop before the string input and after other scanf:
  - while ((c = getchar()) != '\n' && c != EOF); //you should declare c before using it
  - See example code in the next slide

```c
int main()
{
  int id;
  char name[30];
  char ch, c;
  while(1){
    printf("\nMenu, e: enter, x: exit: ");
   scanf("%c",&ch);
    if(ch=='x'){
      printf("\n<<<<<<<EXIT>>>>>>\n\n\n");
      break;
    }
    if(ch=='e'){
      printf("\nEnter student id: ");
      scanf("%d",&id);
      printf("\nEnter student name: ");
      while ((c = getchar()) != '\n' && c != EOF); //added this to eat the \n
      scanf("%[^\n]s",name);
      printf("Your entered %d %s", id, name);
      while ((c = getchar()) != '\n' && c != EOF); //added this to eat the \n
for next input
    }
  }
  return 0;
}
```

Some useful methods for string available in string.h (you will need them very often in this semester)

- char *strcpy(char *dst, char const *src);
  - It copies src string to the dst string
  - It is programmers responsibility to ensure dst has enough space to copy from src.
- char *strcat(char *dest, const char *src)
  - It takes two arguments, i.e, two strings or character arrays, and stores the resultant concatenated string in the first string specified in the argument.
- size_t strlen(const char *str);
  - Get the length of the passed string
- char *strstr(const char *haystack, const char *needle)
  - Search for needle string inside haystack string and returns a pointer to the first occurrence in haystack. Or a null pointer will be returned if needle is not found in haystack.
- int strcmp(char const *s1, char const *s2);
  - Compares two strings. See the following table to understand he returned date

Return Value	Remarks
0	if both strings are identical (equal)
negative	if the ASCII value of first unmatched character is less than second.
positive integer	if the ASCII value of first unmatched character is greater than second.

strcmp test

```
int main()
{
    char str1[] = "abcd", str2[] = "abde", str3[]="abCd",
str4[]="abcd";
    int result;
    result = strcmp(str1, str2);
    printf("strcmp(str1, str2) = %d\n", result);

    result = strcmp(str2, str1);
    printf("strcmp(str2, str1) = %d\n", result);

    result = strcmp(str1, str3);
    printf("strcmp(str1, str3) = %d\n", result);

    result = strcmp(str3, str1);
    printf("strcmp(str3, str1) = %d\n", result);

    result = strcmp(str1, str4);
    printf("strcmp(str1, str4) = %d\n", result)

    result = strcmp(str4, str1);
    printf("strcmp(str4, str1) = %d\n", result)

    return 0;
}
```

```
/*output
strcmp(str1, str2) = -1
strcmp(str2, str1) = 1
strcmp(str1, str3) = 32
strcmp(str3, str1) = -32
strcmp(str1, str4) = 0
strcmp(str4, str1) = 0
*/
```

STRUCTS

Struct review

- When the data you want to store doesn't fit neatly into a predefined C type, you can create your own. Here is an example:

```
struct employee {
  char[20] name;
  double salary;
  int empID;
};
```

- Now, to declare a variable of this new data type, we write:

```
struct employee officeworker;
```

- To access the fields of a struct employee variable do as follows:

```
officeworker.empID = 1;
```

Struct and Pointer

- **Structure is used to group related variables (members) of various types together in an organized way. Also, we call it custom data type.**

```
struct employee {
  char[20] name;
  double salary;
  int empID;
};
```

- **Now, to declare a variable of this declared type, we write:**

```
struct employee officeworker;
```

- **Often times, for efficiency purposes, it's easier to declare pointers to structs and use those to manipulate the structures formed.**
- **Consider the following:**

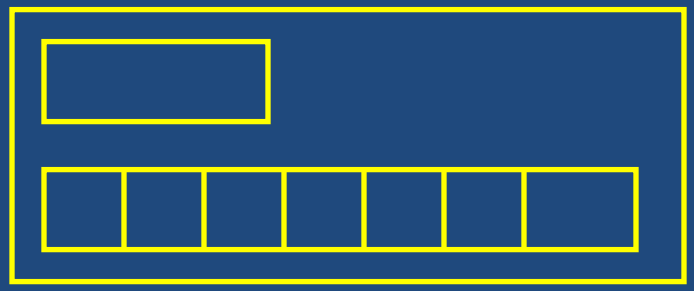
```
struct employee *temp;// declare a pointer of type employee
temp = &officeworker; //store the address of office worker to temp
(*temp).salary = 50000; //dereference temp to access salary of officeworker
```

- **Since expressions similar to the last one are used so often, there is a shorthand to access a field/member of a record through a pointer to that record. The last statement can also be written as:**

```
temp->salary = 50000 // -> is mostly used to dereference structure pointer.
```

- **Another reason to use a pointer to a struct is to dynamically allocate the memory for the struct. (We will briefly discuss about dynamic memory allocation in our next class).**
- **This allows the memory to be allocated beyond the "life"/"scope" of the function within which it was declared.**

Typedef and Pointers and Structures

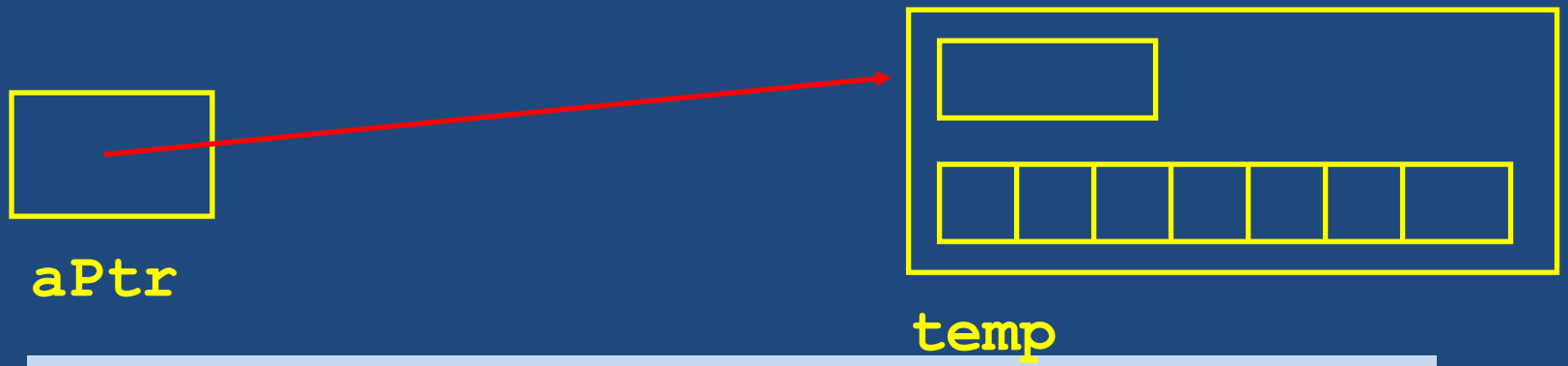



```
typedef struct bookRec{
      float price;
      char name[7];
}Book;
//if you use typedef, you don't have to write struct
every time when you declare a variable of your
structure

Book temp; //this line is possible as we have used typedef

scanf("%d %s", &temp.price, temp.name);
```

Pointers and Structures




```
struct bookRec{
      float price;
      char name[7];
}Book;

Book *aPtr;
Book temp;
aPtr = &temp;

scanf("%d %s", &(aPtr->price), aPtr->name);
```

Structures and Functions

- ***Example 1: Passing individual structure members***

```
struct book{
      char title[40];
      char author[40];
      int price;
    };

void display(char *t, char *a, int p)
{
     printf("\n%s %s %d",t, a, p);
}


int main(){

    struct book b1={"Programming knights","Arup Guha",50};

    display(b1.title, b1.author, b1.price);
}
```

This parameter list has no idea about the structure. It can just take two char pointers and one int

Here, we are not really passing a structure. Just passing individual members

Structures and Functions

- Example 2: Passing entire structure variable

```
struct book{
      char title[40];
      char author[40];
      int price;
    };


void display(struct book b)
{
   printf("\n%s %s %d",b.title,b.author,b.price);
}

main(){
    struct book b1={"Programming knights","Arup
   Guha",50};
     display(b1);
}
```

This function takes a book structure variable.
Just a note: structures are passed by value like other types of variables.

Here, we are passing our structure variable.
Note: we are not passing reerence

Passing reference to a structure in function

```
struct book{
     char title[40];
     char author[40];
     int price;
   };


void display(struct book *b)
{
   printf("\n%s %s %d",b->title,b->author,b->price);
}

main(){
    struct book b1={"Programming knights","Arup
   Guha",50};
     display(&b1);
}
```

This function takes address of a book structure variable. This function has the ability to change the value of b1.

Here, we are passing our structure variable.
Note: we are not passing reerence

Returning a structure from a function

```
struct point{
      int x;
      int y;
};


struct point get()
{
   struct point p;
   scanf("%d %d", &p.x, &p.y);
  return p;
}

main(){
 struct point p1, p2;
 p1 = get();//calling get() function and storing the result in p1
 p2 = get();

}
```

This function returns a point type data

Array of Structures

- Let's consider a case where we want to store data of 100 books. We would be required to use 100 different structure variables from b1 to b100, which is definitely not very convenient.
- A better approach would be to use an array of structures.
- Example:

```
typedef struct Book {
     char title[50];
     char author[50];
     float price;
     int pages;
}Book;

Book books[100];
```

books[0]	title, author, price, pages
books[1]	title, author, price, pages
books[2]	title, author, price, pages
books[3]	title, author, price, pages
……	
……	
Books[99]	title, author, price, pages

This provides space in memory for 100 structure variables of type ***Book***.

Example Array of structure

```c
#include <stdio.h>
#include <string.h>
#define SIZE 3
typedef struct Book {
     char title[50];
     char author[50];
     float price;
     int pages;
}Book;
int main( ) {
     Book books[SIZE];
     char x;

     for(int i=0; i<SIZE; i++) {
          printf("Enter book title for book %d: ",
          i+1);
          scanf("%[^\n]s", books[i].title);
          //clearing up the \n from the last input
          while((x=getchar() != '\n') && x != EOF);
          printf("Enter author for book %d: ", i+1);
          scanf("%[^\n]s", books[i].author);
          printf("Enter price for book %d: ", i+1);
          scanf("%f", &books[i].price);
          printf("Enter total pages for book %d: ",
          i+1);
          scanf("%d", &books[i].pages);
          while((x=getchar() != '\n') && x != EOF);
          //clearing up the \n from the last input
     }
```

Declaring array of structure. As we have used typedef, no need to write struct key word. Otherwise you have to write struct keyword.

Continuation of the code

```c
printf("Displaying the book list:
\n");
for(int i=0; i<SIZE; i++)
{
     printf("Information for book
     %d:\n Name: %s Auhor: %s Price:
     %0.2f Pages: %d\n",i+1,
     books[i].title, books[i].author,
     books[i].price, books[i].pages);
}
return 0;
}
```

Example output of the last slide's code

```
> ./main
Enter book title for book 1: book1
Enter author for book 1: author1
Enter price for book 1: 50
Enter total pages for book 1: 200
Enter book title for book 2: book2
Enter author for book 2: author2
Enter price for book 2: 300
Enter total pages for book 2: 1000
Enter book title for book 3: book3
Enter author for book 3: author1
Enter price for book 3: 60
Enter total pages for book 3: 300
Displaying the book list:
Information for book 1:
 Name: book1 Auhor: author1 Price: 50.00 Pages: 200
Information for book 2:
 Name: book2 Auhor: author2 Price: 300.00 Pages: 1000
Information for book 3:
 Name: book3 Auhor: author1 Price: 60.00 Pages: 300
```

File I/O

- Please go through the File I/O example codes under C review folder in webcourses.
- See how fscanf and fprintf works.
- We will need to use fscanf and fprintf in various codes in this semester.

Source Codes

- All the codes in the slides as well as more examples are available in creview folder in webcourses.